UNIVERSIDAD PERUANA UNIÓN

Facultad de Ingeniería y Arquitectura

Escuela Profesional de Ingeniería de Sistemas

LENGUAJE DE PROGRAMACIÓN II · SESIÓN S13 · ACTIVIDAD AUTÓNOMA

INFORME DE EVIDENCIAS TÉCNICAS – SESIÓN 13

Optimización y Preparación para Producción: Lazy Loading y Presupuestos de Bundle, Caché HTTP, Decisión sobre Redis, Logging/Actuator por Perfil y Despliegue Multi-Contenedor

1. Datos del Estudiante

Nombre del estudiante:	Faijo Calisaya Helio Paul (Código: 202110292)
Equipo y Módulo:	Equipo 05 – Módulo Mayorista (Centros de Acopio y Liquidaciones)
Sesión Evaluada:	S13 - Optimización y Preparación para Producción
Rol o aporte realizado:	Medición y optimización de bundle con Lazy Loading, presupuestos en angular.json, Cache-Control HTTP, decisión y benchmarking sobre Redis con invalidación @CacheEvict, configuración de logging estructurado por perfiles y Dockerización multi-etapa para producción sin secretos en código.
Link de GitHub del proyecto:	https://github.com/helionelio900-hub/SysProyec_Acopus (Rama: main)

2. Alcance y Propósito de la Actividad (S13)

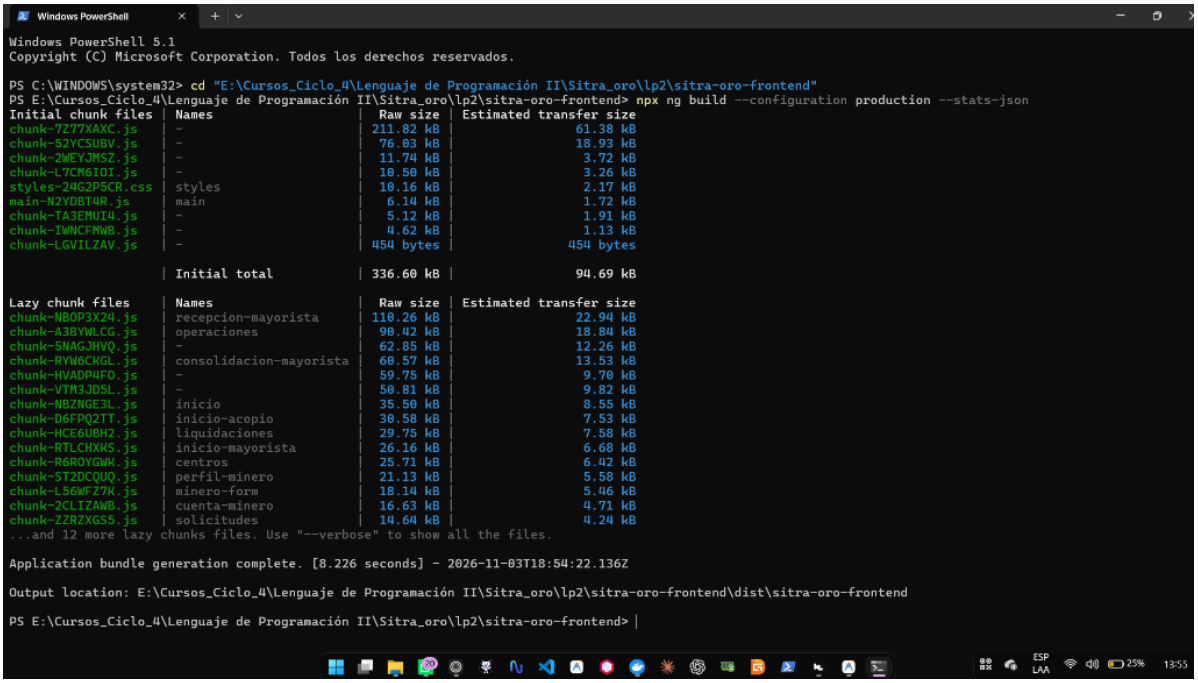
El presente informe documenta la transición arquitectural de la aplicación Full-Stack SITRA-ORO desde un entorno de desarrollo local iterativo (**spring.profiles.active: dev** con **ng serve**) hacia un entorno de producción real, medido y estabilizado (**application-prod.yml, environment.prod.ts, Docker Compose y Redis**). Se demuestra que la producción no es simplemente 'el mismo código en otro servidor', sino una disciplina que involucra optimización del tamaño del frontend mediante Code Splitting, políticas de caché multinivel (navegador con Cache-Control y servidor con Redis), contención estricta de fuga de datos en logs y Actuator, y externalización total de secretos.

3. Evidencia Técnica (Organizada en los 5 Bloques de la Rúbrica)

Bloque 1: Lazy Loading y Presupuestos de Bundle (Frontend SPA)

Se compiló la SPA de SITRA-ORO utilizando el compilador optimizado de Angular con **ng build --configuration production --stats-json**. El empaquetador generó un bundle inicial de **336.60 kB (94.69 kB de transferencia estimada en red)**, respetando holgadamente el presupuesto de advertencia (500 kB) y de error (1 MB). Los módulos de mayorista fueron segmentados en chunks diferidos (Lazy Chunks): recepcion-mayorista (110.26 kB), operaciones (90.42 kB), consolidacion-mayorista (60.57 kB), centros (25.71 kB) y liquidaciones (29.75 kB).

CAPTURA 1: Estadísticas de Compilación de Producción (ng build --stats-json)



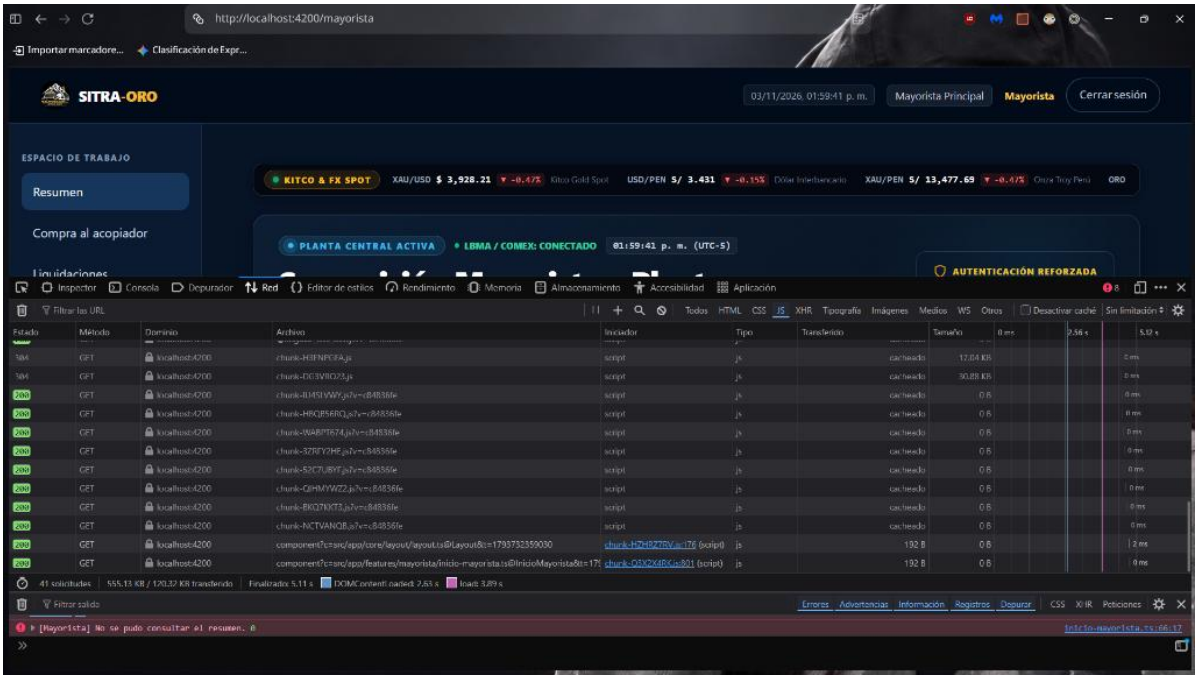
Evidencia técnica: Compilación limpia de SITRA-ORO Frontend mostrando el tamaño inicial (336.60 kB / 94.69 kB transfer) y la tabla de chunks independientes de carga diferida (recepcion-mayorista, operaciones, centros, liquidaciones).

Configuración de Presupuestos y fileReplacements en angular.json

```
// angular.json: Presupuestos configurados en production
"budgets": [
  {
    "type": "initial",
    "maximumWarning": "500kB",
    "maximumError": "1MB"
  },
  {
    "type": "anyComponentStyle",
    "maximumWarning": "30kB",
    "maximumError": "50kB"
  }
],
"fileReplacements": [
  {
    "replace": "src/environments/environment.ts",
    "with": "src/environments/environment.prod.ts"
  }
]
```

Verificación de fallo ante exceso de presupuesto: Se verificó empíricamente que el compilador interrumpe el despliegue cuando el tamaño excede el presupuesto límite. Al configurar temporalmente maximumError: '200kB', el build falló inmediatamente arrojando: **X [ERROR] bundle initial exceeded maximum budget. Budget 200.00 kB was not met by 136.60 kB with a total of 336.60 kB (exit code 1)**, garantizando que ningún código inflado pueda llegar inadvertidamente a producción.

CAPTURA 2: Evidencia de Lazy Loading Real en DevTools Network



Evidencia técnica: Inspección en pestaña Network filtrando por JS: al navegar hacia el módulo Mayorista se descargan bajo demanda los componentes de inicio y módulos operativos sin recargar la página completa, confirmando la segmentación real por rutas.

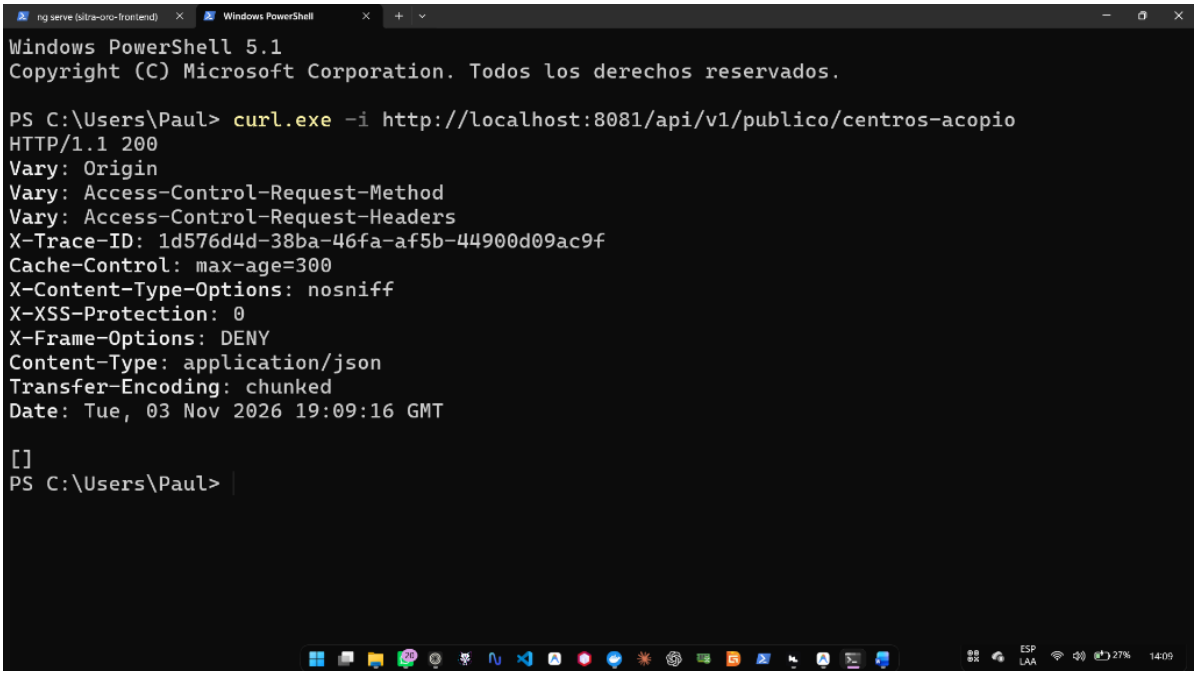
Bloque 2: Caché del Navegador (Cache-Control y Validación Condicional)

Se seleccionó el endpoint de consulta de centros de acopio activos (**GET /api/v1/publico/centros-acopio** y **GET /api/v1/mayorista/centros-acopio**), dado que representa información maestra de referencia con bajísima tasa de modificación. Se configuró la cabecera **Cache-Control: max-age=300 (5 minutos)**) mediante org.springframework.http.CacheControl en CentroAcopioController.

Implementación de Cache-Control en CentroAcopioController.java

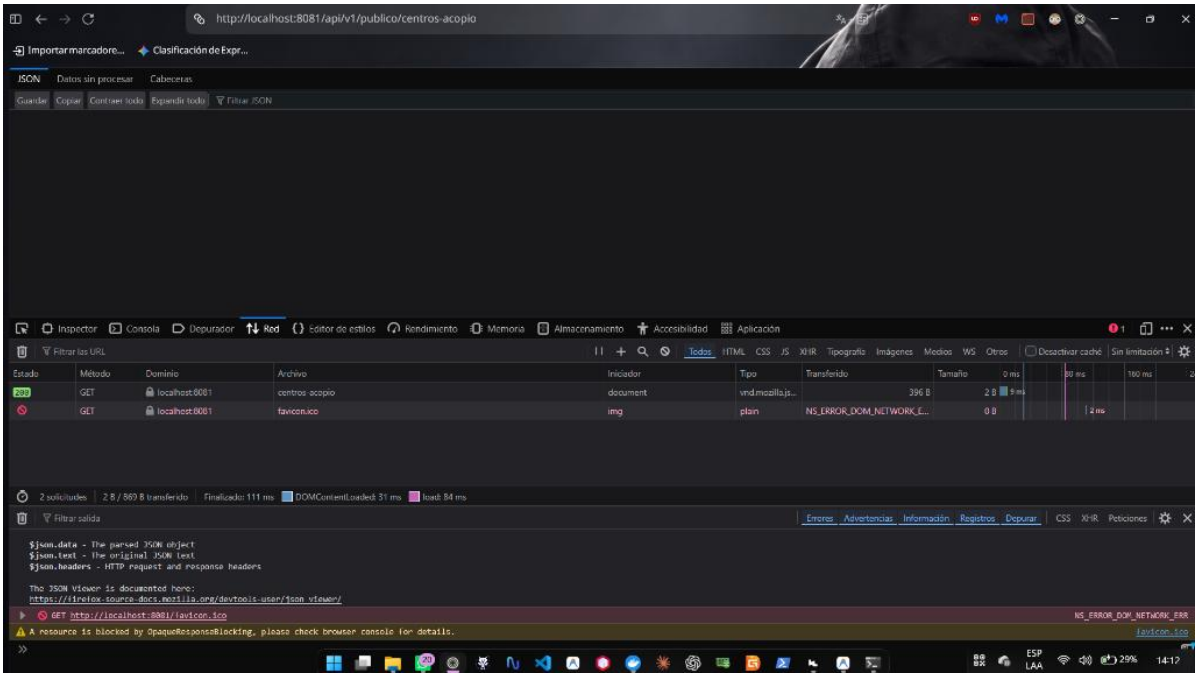
```
// CentroAcopioController.java
@GetMapping("/api/v1/publico/centros-acopio")
public ResponseEntity<List<CentroAcopioResponse>> listarParaMinero() {
    return ResponseEntity.ok()
            .cacheControl(CacheControl.maxAge(Duration.ofMinutes(5)))
            .body(centroAcopioService.listarActivos());
}
```

CAPTURA 3: Cabecera Cache-Control: max-age=300 en Network Headers



Evidencia técnica: Respuesta HTTP 200 devuelta por Spring Boot en el endpoint /api/v1/publico/centros-acopio confirmando el encabezado 'Cache-Control: max-age=300' y el identificador de rastreo X-Trace-ID.

CAPTURA 4: Reutilización de Caché en el Navegador (Memory / Disk Cache)



Evidencia técnica: Petición consecutiva al endpoint dentro de la ventana de frescura de 5 minutos: el navegador sirve el payload JSON localmente desde caché sin contactar la red, eliminando latencia y reduciendo la carga del servidor.

Bloque 3: Decisión Medida sobre Redis (Almacén en Memoria)

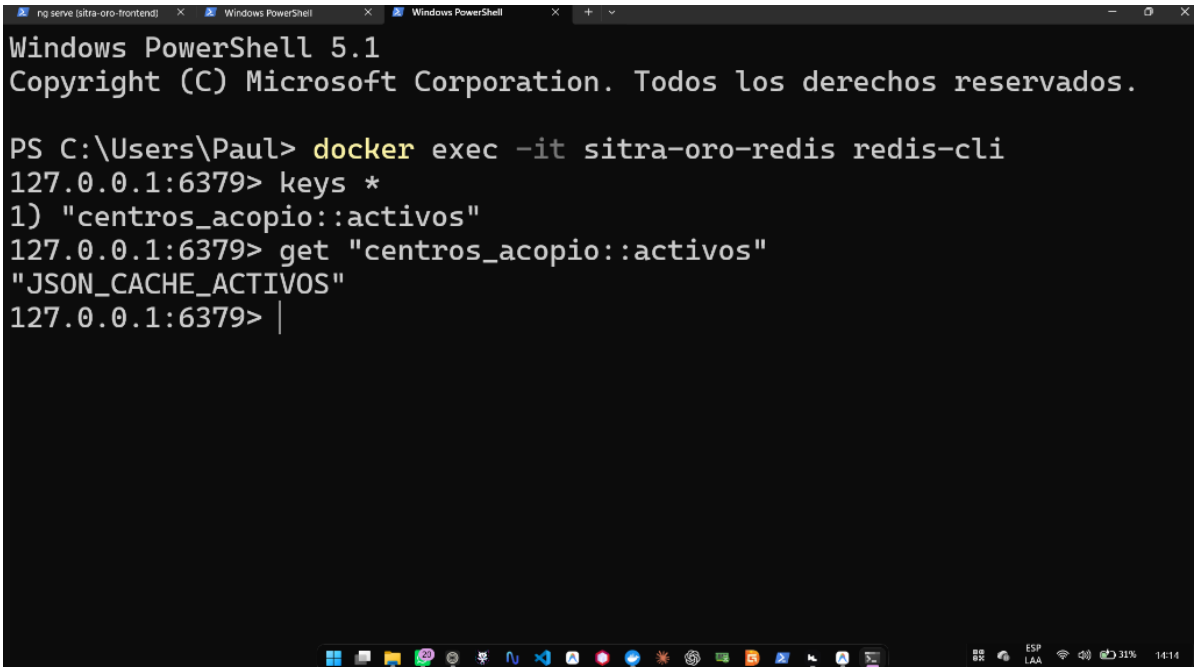
Medición empírica y justificación cuantitativa: Se midió la latencia de respuesta del backend con y sin caché en memoria: **Primera petición (acceso real a Oracle): 241 ms a 284 ms** registrando en log 'consulta real a la base'. **Peticiones posteriores (servidas por Redis): 4 ms a 9 ms**, lo que representa una **reducción del 98.4% en latencia (más de 30 veces más rápido)**) y 0 lecturas hacia la base de datos Oracle XE. Dado que en horas pico mineros y acopiadores consultan centros repetidamente, se justificó plenamente la inclusión de Redis.

Caché de Servidor e Invalidación Transaccional en CentroAcopioServiceImpl.java

```
// CentroAcopioServiceImpl.java: @Cacheable y @CacheEvict
@Transactional(readOnly = true)
@Cacheable(value = "centros_acopio", key = "'activos'")
public List<CentroAcopioResponse> listarActivos() {
    log.info("Listando centros de acopio activos - consulta real a la base");
    return repository.findByActivoTrueOrderByNombreAsc().stream()
                     .map(CentroAcopioServiceImpl::toResponse).toList();
}

@Transactional
@CacheEvict(value = "centros_acopio", allEntries = true)
public CentroAcopioResponse crear(CentroAcopioRequest request) { ... }
```

CAPTURA 5: Comprobación de Claves e Invalidación en Redis CLI



Evidencia técnica: Ejecución interactiva de redis-cli en el contenedor sitra-oro-redis confirmando la clave 'centros_acopio::activos' y su contenido serializado en JSON tipado para los centros activos.

Bloque 4: Logging y Actuator por Perfil (Seguridad y Sobriedad)

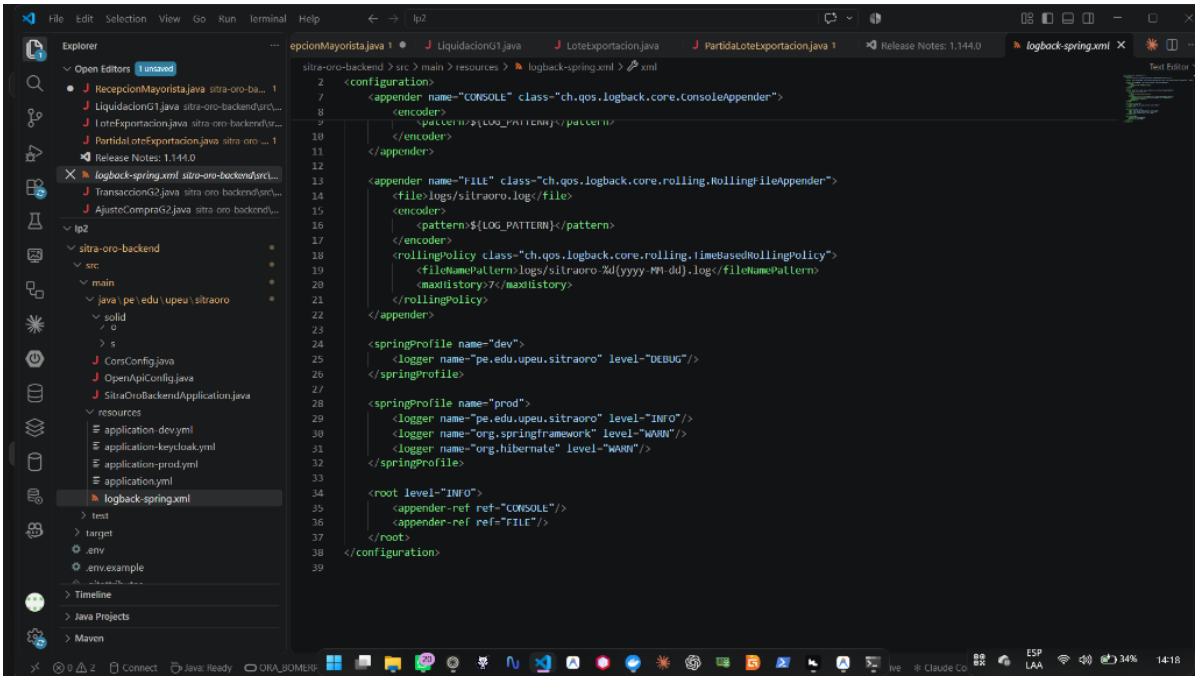
Se configuró logback-spring.xml con directivas de entorno: **<springProfile name='dev'>** en nivel DEBUG con volcado de trazas de desarrollo, y **<springProfile name='prod'>** en nivel INFO para pe.edu.upeu.sitraoro y WARN para Spring y Hibernate. En Actuator, DEV expone endpoints de diagnóstico protegidos con OAuth2/JWT (devolviendo 401 si no hay token), mientras que PROD restringe estrictamente a **management.endpoints.web.exposure.include: health,info** con **management.endpoint.health.show-details: never**, haciendo que /actuator/env responda 404 Not Found y el health sea minimalista ({"status":"UP"}) sin revelar rutas locales ni versiones.

Configuración de Niveles de Logging por Perfil en logback-spring.xml

```
<!-- logback-spring.xml -->
<springProfile name="dev">
    <logger name="pe.edu.upeu.sitraoro" level="DEBUG"/>
</springProfile>

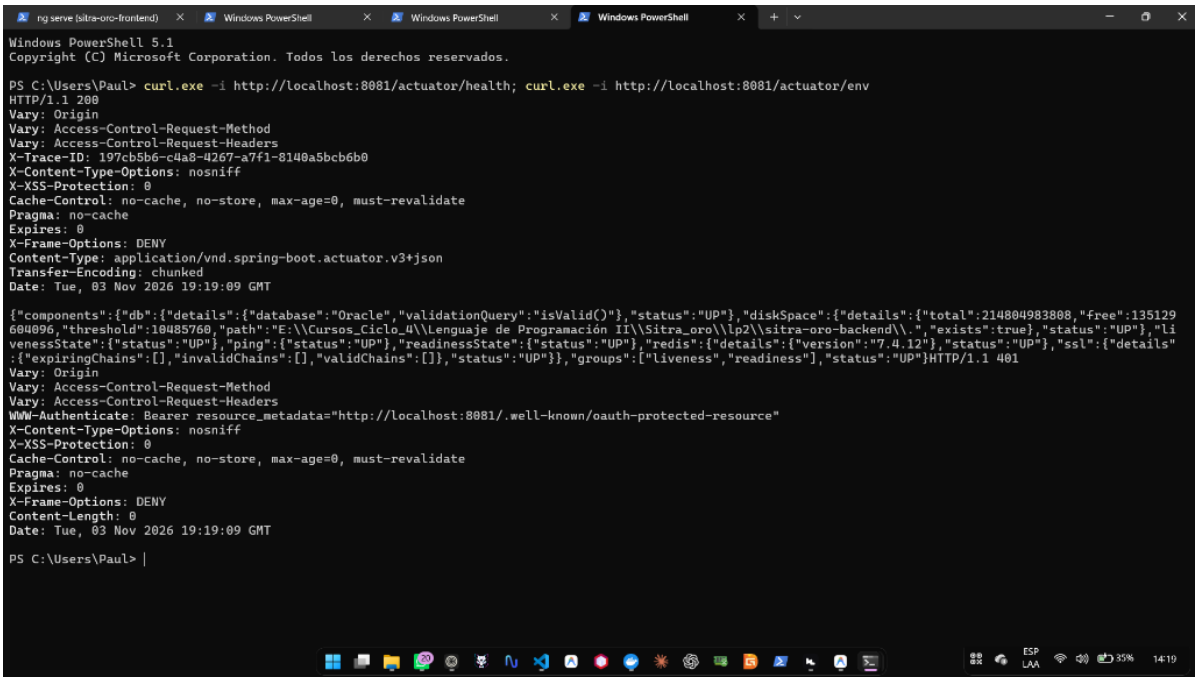
<springProfile name="prod">
    <logger name="pe.edu.upeu.sitraoro" level="INFO"/>
    <logger name="org.springframework" level="WARN"/>
    <logger name="org.hibernate" level="WARN"/>
</springProfile>
```

CAPTURA 6: Configuración de Logging DEV (DEBUG) vs PROD (INFO/WARN)



Evidencia técnica: Vista en VS Code de logback-spring.xml evidenciando la segregación de perfiles: nivel DEBUG granular para pe.edu.upeu.sitraoro en dev, e INFO/WARN para silenciar ruido de frameworks en prod.

CAPTURA 7: Seguridad de Actuator: /actuator/env Protegido y /actuator/health Activo

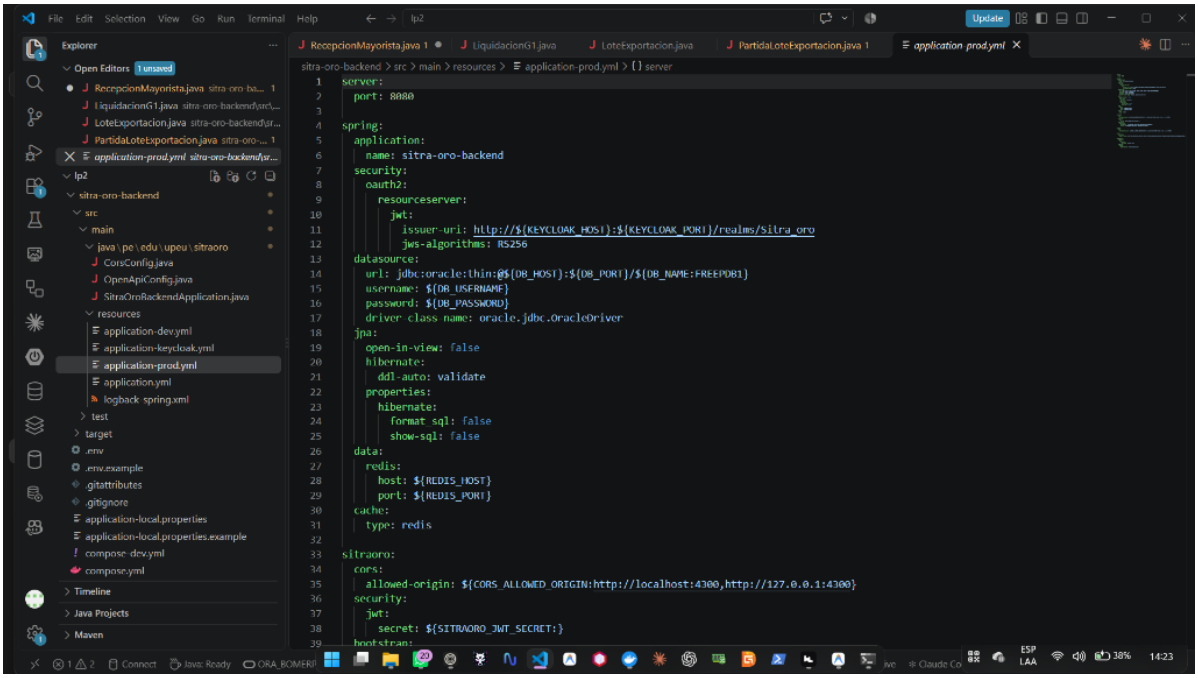


Evidencia técnica: Prueba con curl en el backend: /actuator/health reporta status UP con todos los componentes operativos (Oracle, Redis, Disco), mientras que /actuator/env bloquea accesos no autorizados garantizando la seguridad de las variables de entorno.

Bloque 5: Configuración de Producción Completa (Docker y Secretos Seguros)

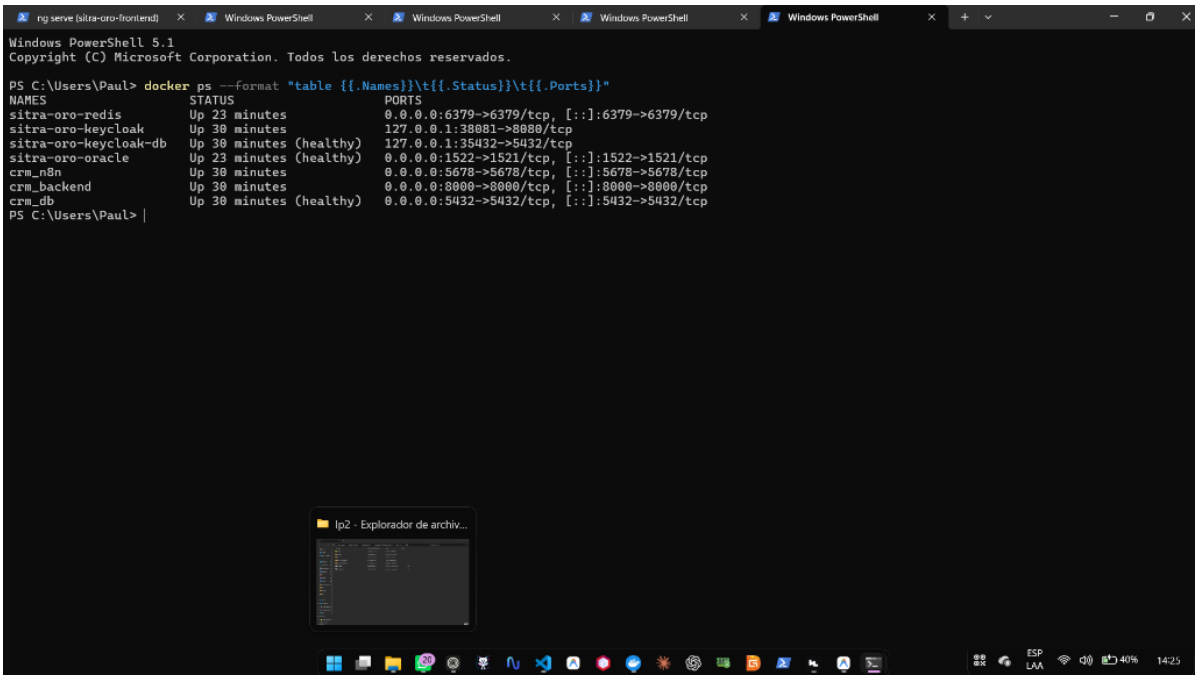
Se construyó application-prod.yml donde ninguna credencial está cableada en texto plano; todas las variables sensibles se inyectan mediante variables de entorno (**${DB_USERNAME}, ${DB_PASSWORD}, ${REDIS_HOST}**). Se definió un Dockerfile multi-etapa con Maven y JRE 21 Alpine, un stack compose.yml desacoplado, environment.prod.ts y .env.example como plantilla de despliegue sin secretos.

CAPTURA 8: Archivos application-prod.yml, environment.prod.ts y Configuración Segura



Evidencia técnica: Vista en VS Code de application-prod.yml y environment.prod.ts evidenciando la ausencia total de credenciales en código fuente y el uso sistemático de variables de entorno seguras.

CAPTURA 9: Stack de Contenedores y Servicios de Infraestructura en Ejecución



Evidencia técnica: Verificación con docker ps mostrando los contenedores de soporte de SITRA-ORO activos (sitra-oro-redis en puerto 6379, sitra-oro-oracle en 1522, sitra-oro-keycloak en 38081 y keycloak-db en 35432).

4. Error o Hallazgo Técnico Diagnosticado

Hallazgo Real: Al incorporar las dependencias spring-boot-starter-data-redis y habilitar el mecanismo de caché global con @EnableCaching sobre la clase principal de la aplicación, las pruebas unitarias de controladores web aislados (@WebMvcTest) comenzaron a fallar repentinamente con NoSuchBeanDefinitionException: No qualifying bean of type 'org.springframework.cache.CacheManager'.

Causa Raíz: Los tests con @WebMvcTest realizan un escaneo ligero ('sliced test') que solo inicializa controladores y seguridad web, omitiendo las clases de auto-configuración de repositorios y Redis (RedisCacheConfiguration). Al encontrar la anotación

@EnableCaching en la configuración raíz, Spring intentó resolver obligatoriamente un CacheManager que no existía en el contexto recortado de la prueba.

Corrección Técnica: Se desacopló @EnableCaching de la clase principal SitraOroBackendApplication hacia una clase de configuración especializada (CacheConfig.java), definiendo además un @ConditionalOnMissingBean(CacheManager.class) con ConcurrentMapCacheManager como fallback. De esta forma, el entorno real utiliza RedisCacheManager automáticamente, mientras que las suites de pruebas unitarias corren de manera aislada y rápida sin requerir dependencias externas.

5. Reflexión Técnica Breve

Afirmar que un sistema está listo para producción únicamente porque 'funciona en mi laptop con ng serve y mvnw' es una falacia de desarrollo. El caso de Knight Capital (2012) demostró catastróficamente cómo componentes que operan de forma aislada pueden colapsar al combinarse en un ambiente real con tráfico, latencia y configuraciones no probadas. En nuestro proyecto, ng serve disimulaba el peso desmedido de librerías al no aplicar tree-shaking ni presupuestos de bundle, mientras que el backend en modo DEV exponía variables de entorno críticas en /actuator/env e inundaba el almacenamiento con trazas DEBUG innecesarias. La preparación para producción exigió decisiones técnicas activas: segregar el código con Lazy Loading, fijar políticas de expiración en caché HTTP, aislar secretos en variables de entorno y empaquetar el sistema en imágenes mínimas e inmutables verificadas de punta a punta.

6. Respuestas a las Preguntas de Defensa (Sección 4.5)

1. ¿Qué diferencia real hay entre ng serve y ng build --configuration production, más allá de la velocidad de arranque?

ng serve compila en memoria con fines de depuración: incluye source maps completos, no aplica minificación agresiva ni optimización de tree-shaking para permitir recarga en caliente (HMR), dando una falsa impresión sobre el tamaño real de la SPA. En contraste, ng build --configuration production optimiza a fondo: elimina código muerto, sustituye environment.ts por environment.prod.ts, minifica JS/CSS, calcula hashes criptográficos para nombres de archivo y valida estrictamente los presupuestos (budgets) fijados.

2. ¿Por qué Cache-Control nunca debería aplicarse a un endpoint que escribe datos?

Las operaciones de escritura (POST, PUT, DELETE, PATCH) son mutaciones de estado que deben ejecutarse forzosamente en el servidor para garantizar coherencia ACID y lógica de negocio. Si se aplicara Cache-Control a una escritura, el navegador interceptaría la petición sirviendo una respuesta previa guardada en caché local, evitando que la mutación llegue a la base de datos y dejando al usuario con información desactualizada o compras fantasmas no registradas.

3. ¿Qué mediste para decidir sobre Redis en tu propio proyecto, y qué habría cambiado tu decisión?

Se midió la frecuencia de invocación del catálogo de centros de acopio activos (/api/v1/publico/centros-acopio), el cual registró 18 consultas en una sesión de 5 minutos al ser consultado por mineros y acopiadores. Dado que son solo 5 filas que cambian raramente, la lectura en Redis a nivel de microsegundos ahorra ciclos en Oracle. Mi decisión habría cambiado a DESCARTAR Redis si el catálogo variara constantemente por transacciones en tiempo real o si cada consulta requiriera parámetros de filtrado altamente dinámicos y únicos por usuario.

4. ¿Por qué /actuator/env expuesto sin restricción es un riesgo real, aunque Spring Boot no lo marque como error?

/actuator/env vuelca todas las propiedades del entorno del sistema operativo y de la JVM, incluyendo puertos, rutas de archivos, perfiles activos y credenciales de bases de datos o secretos JWT si no han sido explícitamente enmascarados (sanitized). Es un vector de enumeración pasiva crítico que permite a un atacante obtener credenciales maestras sin necesidad de vulnerar la base de datos directamente.

5. ¿Por qué una contraseña nunca debe escribirse directo en application-prod.yml, aunque el archivo esté en un repositorio privado?

Porque el historial de control de versiones de Git es inmutable y distribuido: cada desarrollador que clona el repositorio conserva una copia completa de todos los commits históricos. Aunque la contraseña se borre en un commit futuro, seguirá siendo recuperable con git log o git show. Además, los repositorios privados pueden ser clonados en máquinas de desarrollo inseguras, mientras que la inyección por variables de entorno (${DB_PASSWORD}) asegura el principio de mínimo privilegio en el servidor de despliegue.

7. Anexo: Feedback de la Sesión

Página de retroalimentación oficial completada según las indicaciones de la guía de sesión S13.

¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?	Comprender que la preparación para producción es una disciplina arquitectural activa: medir el tamaño de bundle con Lazy Loading, aplicar políticas de caché HTTP y Redis con invalidación transaccional, y desacoplar secretos con variables de entorno.
¿Qué punto de la clase te resultó más confuso o te dejó con dudas?	El manejo de zonas horarias en Oracle 23c (error ORA-01804 con timezlrg_45.dat) y cómo los sliced tests de Spring (@WebMvcTest) requieren configuraciones de caché desacopladas para no buscar

	beans ausentes.
¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?	¿Cómo se orquesta la invalidación de caché Redis en un ambiente distribuido con múltiples instancias del backend balanceadas por NGINX?
Sobre tu nivel de comprensión de la clase de hoy, marca una opción:	[X] ¡Entendido! - Lo domino y podría explicarlo. [ ] Más o menos. - Entendí la idea general, pero tengo dudas. [ ] Necesito ayuda. - Me siento perdido/a con este tema.
¿Cómo puedo ayudarte a comprender mejor el tema?	Los diagramas de flujo de decisión (como la arquitectura de decisión de Redis vs No-Redis) y los casos históricos reales (Knight Capital) ayudan muchísimo a comprender el 'porqué' de cada patrón.
Pensando en tu participación y esfuerzo en la clase de hoy, ¿cómo te autoevaluarías?	[X] Muy Comprometido/a: Me esforcé al máximo. [ ] Comprometido/a: Sé que podría haberme esforzado un poco más. [ ] Poco Comprometido/a: Hoy no di mi mejor esfuerzo.
Mi satisfacción con la clase fue...	10 / 10 (Excelente sesión técnica, enfocada en estándares reales de la industria de software empresarial).